# LAB 7.2 predictions

## Beskrivelse

Make predictions
We will now implement the button to make predictions.
The predict button is currently disabled. Add the following code at the end of the `train` function to enable it when trained:

```
document.getElementById("predict-button").removeAttribute("disabled");
```

Also add the above line to the load function (after loading the model).
Adapt the `normalise` function to be able to pass in min and max values:

```
function normalise(tensor, previousMin = null, previousMax = null) {
  const min = previousMin || tensor.min();
  const max = previousMax || tensor.max();
  const normalisedTensor = tensor.sub(min).div(max.sub(min));
  return {
    tensor: normalisedTensor,
    min,
    max
  };
}
```

Implement the `predict` function:

```
const predictionInput = parseInt(document.getElementById("prediction-input").value);
if (isNaN(predictionInput)) {
  alert("Please enter a valid number");
}
else {
  tf.tidy(() => {
    const inputTensor = tf.tensor1d([predictionInput]);
    const normalisedInput = normalise(inputTensor, normalisedFeature.min, normalisedFeature.max);
```

## Oplysninger

Udgivet 23. marts 2026 af Morten Sabro Damgaard

Denne arbejdsopgave er obligatorisk

```
    const normalisedOutputTensor =
model.predict(normalisedInput.tensor);
    const outputTensor = denormalise(normalisedOutputTensor,
normalisedLabel.min, normalisedLabel.max);
    const outputValue = outputTensor.dataSync()[0];
    document.getElementById("prediction-output").innerHTML
= `The predicted house price is: <br />`
      + `<span style="font-size:
2em">\$${(outputValue/1000).toFixed(0)*1000}</span>`;
  });
}
```

Visualise the prediction line

We will extend our plot to visualise the prediction line:

Adapt the `plot` function to take a second optional series for prediction:

```
async function plot (pointsArray, featureName,
predictedPointsArray = null) {
  const values = [pointsArray];
  const series = ["original"];
  if (Array.isArray(predictedPointsArray)) {
    values.push(predictedPointsArray);
    series.push("predicted");
  }

  tfvis.render.scatterplot(
    { name: `${featureName} vs House Price` },
    { values, series },
    {
      xLabel: featureName,
      yLabel: "Price",
    }
  );
}
```

Add the `plotPredictionLine` function which will prepare a prediction line and plot it:

```
async function plotPredictionLine () {
  const [xs, ys] = tf.tidy(() => {

    const normalisedXs = tf.linspace(0, 1, 100);
    const normalisedYs =
model.predict(normalisedXs.reshape([100, 1]));
```

```
    const xs = denormalise(normalisedXs,
normalisedFeature.min, normalisedFeature.max);
    const ys = denormalise(normalisedYs, normalisedLabel.min,
normalisedLabel.max);

    return [ xs.dataSync(), ys.dataSync() ];
  });

  const predictedPoints = Array.from(xs).map((val, i) => {
    return {x: val, y: ys[i]}
  });

  await plot(points, "Square feet", predictedPoints);
}
```

Create a global variable for `points` above the `run` function and remove the `const` keyword to use it within the method.
We want to plot the prediction line after training, so call `plotPredictionLine` in the `train` function (after calling train):

```
await plotPredictionLine();
```

We also want to plot the prediction line after loading a model. Call `plotPredictionLine` (as above) in the `load` function (after loading).
We also want to plot the prediction line during training so we can visualise the training of the model. We can do this by plotting the line with the current model parameters at the beginning of each epoch. Add within `onEpochBegin` (in `trainModel`).